AFRINEXT · PHASE 3 · DESIGN GATE — NO CODE WRITTEN

Refund Execution

A buyer whose verified payment ended as `refund_due` must get their money back **exactly once**, with evidence. This is the design for that, and the decisions I will not take on my own.

Branch **claude/create-app-repository-1cxsih** | Designed against **4342c06**

Design only · no implementation | Decisions needing you **7**

WHAT THIS DOCUMENT IS PINNED TO

Code state 4342c06 — the commit you approved at the Phase 2 financial gate. Every "state, verified at" claim below was read from that tree. This document and its PDF are committed *above* that commit, so the branch tip is newer than `4342c06` — but every commit between them touches `docs/` only, which is why the code state named here stays exactly right. Verify with `git diff --stat 4342c06 HEAD -- . ':!docs'`: it is empty.

THE SINGLE HARDEST PROBLEM IN THIS PHASE

Refunding twice is worse than refunding late. Every design choice below is subordinate to one question: **when we do not know whether the provider executed a refund, what do we do?** A timeout is not evidence of failure. The system must be able to say *"unknown"* and act on it — and nothing may treat unknown as "try again".

A The architecture a refund lands in

THING THAT EXISTS	STATE, VERIFIED AT 4342C06
`orders.status = 'refund_due'`	Reachable, indexed, and reached only from a *verified* provider event. Terminal today.
`payments`	Holds the authoritative `amount_minor`, `currency`, `provider`, `provider_ref`. **This is the financial snapshot a refund must copy from.**
`PaymentProvider.refund()`	**Exists** — takes a charge ref, Money, reason and an idempotency key; returns `pending | succeeded | failed`.
`PaymentProvider.getRefund()`	**DOES NOT EXIST** There is no way to ask a provider what became of a refund. This is the gap that makes ambiguity unresolvable, and §E fixes it.
`VerifiedEvent`	Charge-shaped: `status` is a `ChargeStatusValue` and `providerRef` means a charge. **A refund webhook cannot currently be represented.**
`payment_events`	Append-only, `UNIQUE (provider, provider_event_id)` — the replay defence, reusable for refunds.
`Ledger`	`refund` is a declared transaction kind; **no refund flow function exists**, and Phase 2 posts *nothing* to the ledger. See §Q.
`authorize() · order.refund`	Permission exists, held by `finance` only. `support` has `ledger.read` and no money capability.
`Actor.elevated`	Step-up flag exists on the actor but **nothing enforces it anywhere yet**.
`reconcileCommerce()`	Five detectors, CI-gated, and CI proves the detector can still report a fault.
`MessageSender`	A provider-neutral send interface — but immediate-send, not an event. No SMS provider chosen.
`Workers / schedulers`	**NONE EXIST** `expireLapsedOrders()` is a function nobody calls on a schedule. §L and decision 5.
`Admin UI`	None. Only the `/api/v1/admin/ledger` route. An operator queue is new work.

B

Files inspected

`payments/provider.ts` · `payments/mock.ts` · `payments/ipaymoney.ts` · `payments/registry.ts` · `orders/state.ts` · `orders/payment.ts` · `orders/checkout.ts` · `ledger/flows.ts` · `ledger/post.ts` · `ledger/reconcile.ts` · `ledger/commerce-reconcile.ts` · `authz/permissions.ts` · `authz/authorize.ts` · `audit/index.ts` · `auth/messaging.ts` · `db/schema/commerce.ts` · `db/schema/ledger.ts` · migrations `0009` – `0011` · `scripts/reconcile-check.ts` · `api/v1/admin/ledger/route.ts` · `docs/providers/ipaymoney/README.md` · `docs/architecture/late-payment-policy.md` · the live schema for `payments`, `payment_events`, `orders` · all 22 domain test files and 7 browser specs.

C Proposed refund state machine

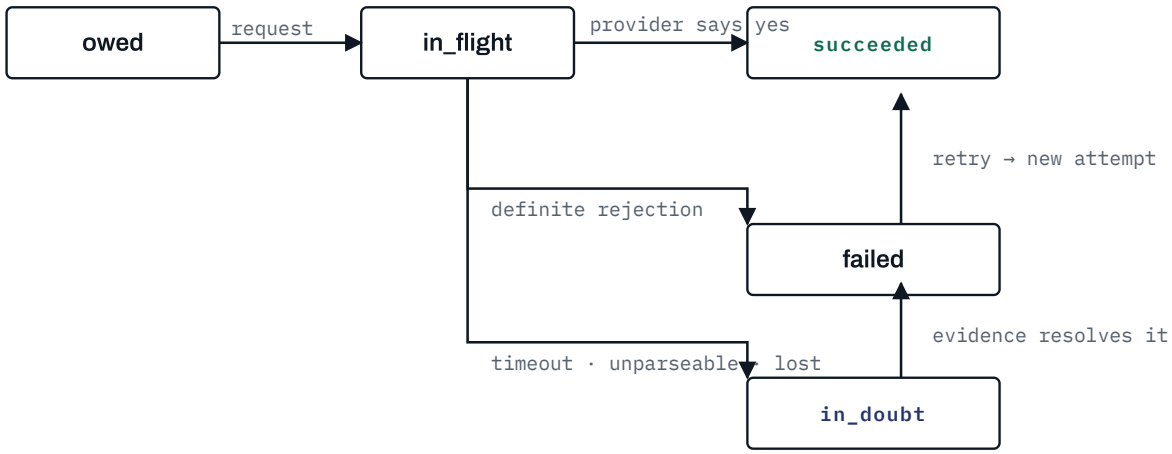
Five states. The names differ from your sketch in one important way, and that difference is the whole design.

STATE	MEANING	TERMINAL
`owed`	A refund record exists because an order reached `refund_due`. Nothing has been sent to any provider.	no
`in_flight`	An attempt has been **recorded as about to be sent**. Written *before* the provider call, so a crash mid-call still leaves a trace.	no
`succeeded`	The provider gave positive evidence: a success response, or a verified refund webhook, or a status query returning success.	**yes**
`failed`	The provider gave a **definite** rejection. Retryable by an operator; a retry opens a new attempt, not a new refund.	no
`in_doubt`	**We do not know.** Timeout, unparseable response, connection lost, or a status query that cannot decide. `NO AUTOMATION MAY ACT ON THIS STATE.`	no — human only

WHY IN_DOUBT IS A FIRST-CLASS STATE AND NOT AN ERROR

Your brief says a timeout must not be treated as proof the refund did not happen. The only way to hold that line in code is to give "we do not know" somewhere to live. If ambiguity collapses into `failed`, retry logic will eventually double-refund; if it collapses into `succeeded`, a buyer is told they were paid and was not.

`in_doubt` is resolved by **evidence only**: a status query that answers definitively, or a verified webhook. Never by a timer, and never by an operator's opinion — see §H on the manual path.



Only `succeeded` is terminal. `failed` and `in_doubt` are open states with an owner: a person.

A retry moves `failed → in_flight` and creates a **new attempt row**; the refund record itself is never duplicated. `in_doubt` leaves only on evidence — a status query or a verified webhook — never on a timer.

TRANSITION	WHO MAY CAUSE IT	GUARD
`— → owed`	system, when an order becomes `refund_due`	one refund per payment (unique index)
`owed → in_flight`	operator with `refund.execute`, or the processor	guarded `UPDATE … WHERE state = 'owed'`, row locked
`in_flight → succeeded`	provider evidence only	guarded; attempt must exist
`in_flight → failed`	definite provider rejection	guarded
`in_flight → in_doubt`	timeout / unparseable / crash recovery	guarded
`failed → in_flight`	operator retry, or scheduled retry within policy	attempt cap; backoff elapsed
`in_doubt → succeeded / failed`	**evidence only** — status query or verified webhook	guarded; evidence recorded
`in_doubt → in_flight`	FORBIDDEN re-sending a refund we cannot account for is the double-refund bug	—

D Proposed schema — two tables

`refunds` — one row per owed refund, holding the frozen money and the current state.

COLUMN	WHY IT EXISTS
`id`	uuidv7
`payment_id → payments`	**UNIQUE.** One refund per payment — the structural guarantee against a second refund record.
`order_id → orders`	Which order created it. "Why is this owed?" answered by join.
`buyer_user_id → users`	Who is owed. Denormalised deliberately: the operator queue must not need three joins to answer "whose money".
`amount_minor · currency`	**Copied from `payments` at creation and never recomputed.** bigint minor units. Never derived from a product price.
`provider · charge_provider_ref`	Which provider owes it, and against which charge.
`reason`	The machine reason from the late-payment decision: `grace_window_elapsed`, `already_owned`, `product_unpublished`, `store_unpublished`.
`state`	CHECK-constrained to the five states.
`attempt_count`	Cheap read for the queue and the retry cap.
`next_attempt_at`	Backoff schedule. NULL means "not scheduled".
`created_at · requested_at · settled_at`	Age, when first sent, when it ended.
`last_error_code · last_error_message`	For the operator. **Never raw provider payloads** — those live in the attempt row.

`refund_attempts` — append-only, one row per provider call. This is the evidence log.

COLUMN	WHY IT EXISTS
`id · refund_id`	—
`attempt_no`	UNIQUE with `refund_id`.
`idempotency_key`	**UNIQUE.** Derived, not random: `refund:{refund_id}:{attempt_no}`. Sent to the provider.
`started_at`	Written **before** the provider call.
`outcome`	`succeeded | failed | in_doubt`, or NULL while genuinely in flight.
`provider_refund_ref`	What the provider called it. NULL when the call never returned one.
`response_code · response_body`	Truncated, secrets never included. §J.
`resolved_by`	`response | status_query | webhook | manual_reconciliation` — **how we know**, which reconciliation checks.

Refund webhooks reuse `payment_events` — its `UNIQUE (provider, provider_event_id)` is already the replay defence, and a second events table would be a second replay bug waiting to happen. It needs a nullable `refund_id` column and its `payment_id` stays as the charge link.

E Provider interface — two additions, no iPayMoney

The existing `refund()` stays. Two things are missing and both are required for safety, not convenience:

1 `getRefund(providerRefundRef)`

Without it, `in_doubt` can never be resolved except by a human phoning the provider. It is the single most important addition in this phase. **Optional on the interface**, exactly as `createPayout` is: if a provider has no refund status endpoint, that is a fact about the provider, and the code must handle its absence rather than assume.

2 A refund shape for verified events

`VerifiedEvent` is charge-shaped. Refund webhooks need either a discriminated union (`kind: 'charge' | 'refund'`) or a separate `verifyRefundWebhook`. I recommend the union: one signature check, one replay table, one code path.

> **WHAT THE MOCK MUST GAIN, AND WHAT THAT DOES NOT PROVE**
>
> The mock needs to be able to answer *slowly*, *ambiguously* and *not at all* — a timeout, a malformed body, a success-after-timeout — because those are the cases the design exists for. Deterministic, driven by the refund reference, exactly as charge outcomes are today.
>
> **None of this proves anything about iPayMoney.** It proves our boundary holds under adversarial provider behaviour. §N lists what is still unknown.

F Idempotency — the crash window

Four independent layers, because each one covers a case the others cannot:

LAYER	STOPS
`UNIQUE (payment_id)` on `refunds`	Two refund records for one payment — duplicate operator action, duplicate event.
Guarded `UPDATE … WHERE state = ?` + row lock	Two workers moving the same refund.
Attempt row written **before** the call, with a derived idempotency key	The crash window. A process that dies mid-call leaves an attempt with no outcome.
`UNIQUE (provider, provider_event_id)` on `payment_events`	Webhook replay.

THE CASE THAT DECIDES THE DESIGN: CRASH BETWEEN THE CALL AND THE WRITE

The provider may have executed the refund. We have an attempt row with `started_at`, an idempotency key, and no outcome. **The recovery rule is: never re-send. Resolve.**

On restart, an attempt older than a threshold with no outcome moves its refund to `in_doubt`, and resolution is attempted with `getRefund()`. If the provider has no status endpoint, or answers unhelpfully, the refund *stays* `in_doubt` and appears in the operator queue. It does not retry. It does not fail. It waits for a person, which is the correct behaviour when money may already have moved.

Re-sending with the same idempotency key would be safe *if* the provider honours idempotency keys — and that is a provider fact we do not have for iPayMoney (§N, item 10). Designing on an unverified assumption about someone else's API is how double refunds happen.

G Concurrency

Same instrument as the late-payment gate, which is already proven: `SELECT … FOR UPDATE` on the refund row inside the transaction that decides, plus guarded transitions. For a multi-worker processor, `FOR UPDATE SKIP LOCKED` so workers take disjoint work rather than queueing behind each other.

> **THE TEST WILL HOLD THE LOCK, NOT RACE TWO PROMISES**
>
> Phase 2 taught this directly: a `Promise.all` concurrency test passed with and without the row lock, because two promises do not reliably interleave two transactions. The refund tests will take the lock on a **second connection**, drive the other worker into it, and prove the outcome — deterministically, every run. Removing the lock must fail that test.

H Authorization and operator safety

CAPABILITY	PERMISSION	HELD BY
See the refund queue	`refund.read`	finance, support
Request execution / retry	`refund.execute`	finance only
Cancel a refund	`refund.cancel`	NOT PROPOSED — see below
Record manual reconciliation	`refund.reconcile_manual`	finance, **with step-up**, and never the same person twice — see decision 4
See one's own refund	`order.read_own`	the buyer, on their own order page only

THERE IS NO "MARK AS REFUNDED" BUTTON, AND THERE SHOULD NOT BE

Your brief asks for no dangerous shortcut, and I agree so strongly that I would rather leave the queue stuck. A refund becomes `succeeded` through provider evidence. The one exception is a **formally defined manual reconciliation**: money returned out of band — a bank transfer, a counter payment — recorded with a mandatory external reference, a reason, step-up re-authentication, and an audit row naming the human. It is a different `resolved_by` value (`manual_reconciliation`) so reconciliation and any auditor can always separate "the provider told us" from "a person told us".

Cancel is not proposed at all. A refund is owed because money was taken and nothing was delivered. Cancelling it means deciding not to give someone their money back, which is not an operator action — it is a dispute, and it needs a policy this phase does not have.

I Reconciliation — every rule you listed, plus one

DETECTOR	CATCHES
`refund_succeeded_without_evidence`	A succeeded refund with no attempt whose `resolved_by` is set. **The most important one.**
`refund_without_refund_due`	A refund whose order is not `refund_due` (or its approved successor state).
`refund_amount_mismatch`	`refunds.amount_minor` ≠ the payment's amount.
`refund_currency_mismatch`	Currency differs from the payment's.
`duplicate_successful_refund`	Two succeeded refunds for one payment. Unreachable by the unique index — which is exactly why it is checked.
`refund_due_without_refund_record`	An order owed a refund that nobody created a record for. **The queue silently losing an item.**
`multiple_active_attempts`	More than one attempt with no outcome for one refund.
`refund_for_missing_payment`	A refund whose payment is absent or not `succeeded`.
`stuck_in_flight`	*My addition.* `in_flight` for longer than the threshold — a crashed worker leaving money in limbo. Not an impossible state; an **unattended** one, which for a refund queue is the same problem.

Each detector gets the Phase 2 treatment: deliberately broken, proven to fire, and the CI gate proves the detector suite can still report a fault rather than passing vacuously.

J Audit events

Actions: `refund.owed_recorded` · `refund.execution_requested` · `refund.attempt_started` · `refund.succeeded` · `refund.failed` · `refund.in_doubt` · `refund.resolved_by_query` · `refund.resolved_by_webhook` · `refund.manually_reconciled` · `refund.retry_scheduled`.

Each carries actor and kind, refund / order / payment ids, previous and new state, amount and currency, provider, provider refund ref, reason, and timestamp — using the existing `before` / `after` columns rather than inventing a shape.

> **WHAT MUST NEVER ENTER THE AUDIT LOG OR THE OPERATOR SCREEN**
>
> Provider API keys, HMAC secrets, webhook signing secrets, authorization headers, full raw provider payloads that may embed credentials, and the buyer's full phone number. The queue shows the buyer by user id and a masked identifier, consistent with the existing privacy posture.

K Retry and failure model

OUTCOME	CLASSIFIED AS	RETRY
Provider rejects with a definite business error	`failed`	Operator only. Automatic retry would hammer a "no".
HTTP 5xx, connection refused, DNS failure	`failed`, retryable	Automatic, with backoff — the call demonstrably did not land.
Timeout after the request was sent	`in_doubt`	NEVER AUTOMATIC
2xx with an unparseable or unrecognised body	`in_doubt`	NEVER AUTOMATIC
Attempt row with no outcome after a threshold	`in_doubt`	resolve via `getRefund()`; otherwise a person

Backoff and caps are `platform_settings`, following the OTP-policy precedent, and **clamped** the way the late-payment grace window is: an operator may make retries less aggressive without asking anyone; making them more aggressive is a code change and a review. Proposed defaults: 1 min, 5 min, 30 min, 2 h, 12 h, then stop at **5 attempts** and hold for manual review.

L Operator workflow and processing

A queue at `/[locale]/admin/refunds`, gated on `refund.read`, showing: refund id, order, payment, buyer (id + masked identifier), amount and currency, reason, age, state, attempt count, last error code and message, next retry time, and how the last outcome was established. Sorted oldest first, because age is the thing that matters to the person waiting.

> **THERE IS NO WORKER INFRASTRUCTURE IN THIS REPOSITORY AT ALL**
>
> Not for refunds, not for expiring checkouts. Building a resilient background worker is a substantial piece of infrastructure, and it is **not** refund logic. Decision 5 offers three ways to process the queue, from "an operator presses a button" to "a real worker loop", and I recommend the middle one.

M Notification boundary

No SMS provider is chosen and none will be invented. The refund domain will **not** call `MessageSender`: that interface sends immediately, and an inline send inside a financial transaction is a way to fail a refund because an SMS gateway was down.

Instead, state changes write a **notification outbox row** — event type, subject, recipient user id, payload — that a future notifier consumes. That keeps Phase 3 independent of the provider decision while making the buyer notification a small later change rather than a redesign. If you would rather not add an outbox table now, the audit log already records every transition and a notifier could read it; I think that is worse, and decision 6 puts it to you.

N iPayMoney — what is still required

> STATED PLAINLY
>
> **No iPayMoney integration will be written in this phase.** The repository contains no official documentation, the provider's domain is unreachable from this environment, and a merchant account is not an API contract. The adapter will continue to throw.

Beyond the twelve items already listed in `docs/providers/ipaymoney/README.md`, refund execution specifically needs:

#	REQUIRED FACT	WHAT IT BLOCKS
1	Refund endpoint, request and response schema	The adapter cannot be written at all
2	**Whether refunds are asynchronous**, and if so the webhook payload and event types	Whether `succeeded` ever arrives synchronously
3	**A refund status query endpoint**, and its complete status enumeration	`in_doubt` is unresolvable without it — every ambiguous refund becomes manual
4	**Whether idempotency keys are honoured on refunds**, and for how long	Whether a retry after a timeout is ever safe
5	Partial refund support and constraints	Whether full-only is a design choice or a provider limit
6	Time limit after capture beyond which a refund is refused	Whether an old `refund_due` can ever be paid at all
7	Refund fees, and who bears them	Whether the buyer receives the full amount
8	Settlement timing for refunds — when the buyer actually sees the money	What we can honestly tell the buyer
9	Error code enumeration, and which are permanent versus transient	The `failed` / `in_doubt` classification in §K
10	Sandbox credentials that can execute a test refund	Any claim that the integration works

Item 3 and item 4 are the two that change the design. If iPayMoney has no refund status query and does not honour idempotency keys, then every timeout becomes a permanent manual case, and you should know that before launch rather than after.

0 Test and mutation matrix

#	SCENARIO	ASSERTED OUTCOME
1	Normal success	`succeeded`, one attempt, evidence recorded, order updated
2	Definite provider rejection	`failed`, error captured, no automatic retry
3	**Timeout**	`in_doubt` — never `failed`, never retried
4	Success arriving *after* a timeout, via status query	`in_doubt → succeeded`, exactly one refund
5	Retry after transient network failure	New attempt, new attempt_no, same refund row
6	Duplicate execution request	Second is a no-op
7	Webhook replay (same event id)	Ignored, one effect
8	Different event id, same outcome	Stale, ignored
9	**Two workers, real row lock held on a second connection**	Exactly one provider call
10	**Crash between provider call and DB write**	Attempt row exists; recovery yields `in_doubt`, **never a re-send**
11	Refund an already-succeeded refund	Refused
12	Wrong currency in provider response	Refused, recorded as evidence mismatch
13	Wrong amount in provider response	Refused
14	Invalid / unparseable provider response	`in_doubt`
15	Missing provider refund reference on success	Refused — success without a reference is not evidence
16	Refund for a nonexistent payment	Refused
17	Refund where the order is not `refund_due`	Refused
18	Second refund record for one payment	Refused by unique index
19	Unauthorized operator (member, support, seller)	403 at the domain, not the screen
20	Manual reconciliation	Requires reference + step-up; audited; `resolved_by = manual_reconciliation`
21	Each of the nine reconciliation detectors	Broken state created, detector fires
22	Browser: buyer sees refund state, cannot trigger one	Real journey through real APIs
23	Browser: operator queue visible to finance, 404/403 to others	Server-enforced

Mutations (each must fail a test): remove the `UNIQUE (payment_id)`; remove the row lock; write the attempt row *after* the provider call instead of before; treat timeout as `failed`; allow `in_doubt →`

`in_flight`; drop the amount check; drop the currency check; drop the `resolved_by` requirement for `succeeded`; remove `authorize()` from execute; loosen each reconciliation condition.

P Security risks

These are the ways a refund path is attacked or abused. Each row is a risk I can name today and the specific thing in this design that answers it — not a promise to be careful.

SECURITY RISK	MITIGATION IN THIS DESIGN
An operator fabricates a success — marks money returned that never left	There is no "mark as refunded" button anywhere in this design. The only human resolution is manual reconciliation, which requires an external provider reference, step-up elevation, and records `resolved_by = manual_reconciliation` — a distinct value reconciliation can single out and finance can re-audit against a provider statement.
Refund redirected to an attacker	No destination is accepted from any caller, ever. A refund is expressed only as "reverse this charge"; the provider returns the money along the path it arrived on. There is no field an API request could set.
Amount or currency tampering	Both are copied from the `payments` row inside the creating transaction and are immutable afterwards. A reconciliation detector compares them to the payment on every run, so a direct database edit is caught rather than trusted.
Forged or replayed refund webhook	The same boundary as charges: signature verified over the raw request bytes before parsing, then the event id checked against the replay index. A refund webhook is not a second, weaker door.
Authorization bypass by direct API call	Every route goes through `authorize()` against `refund.read`, `refund.execute` or `refund.reconcile_manual`, with the actor scoped in SQL. UI absence is never the control — a mutation test removes the check and a test must fail.
Provider credentials or secrets leaking into records	Provider responses are truncated and screened before storage; credentials are never written to `refund_attempts`, never to the audit log, and never rendered in the operator queue.
The processing endpoint reachable by anyone	If decision 5 lands on **B**, the scheduler endpoint authenticates as a machine actor and carries the same permission check as a human — being a cron does not grant it more.

> **THE RISK THAT IS NOT ON THIS TABLE**
>
> Nothing here is verified. This is a design; no code exists to attack yet. Every row above becomes a claim only once the mutation in §O for it has been run and shown to fail a test.

Q Financial risks

These are the ways the money itself goes wrong, independently of whether anyone attacked anything.

FINANCIAL RISK	MITIGATION IN THIS DESIGN
Double refund — paying a buyer twice. The worst outcome in this phase, and the reason for most of the design	Four independent layers: `UNIQUE (payment_id)` so one payment can only ever have one refund; the row lock so two operators serialise; the guarded transition so a second execute finds the wrong state; the attempt row written *before* the provider call so a crash mid-flight leaves evidence. On top of all four, the absolute rule that `in_doubt` is never re-sent — only resolved.
A timeout misread as "it did not happen"	A timeout resolves to `in_doubt`, never `failed`. `in_doubt → in_flight` is forbidden by the state machine, so no code path — and no operator click — can turn an unknown into a retry.
A refund owed and never paid — the queue silently starves	The `refund_due_without_refund_record` and `stuck_in_flight` detectors run in the same CI gate as ledger reconciliation, and the operator queue is age-sorted so the oldest debt is the first thing visible.
Refunding a payment that never actually succeeded	A refund can only be created from a `succeeded` payment row, which itself only exists behind the verified-event boundary from Phase 2. A reconciliation detector re-checks the link.
Minor-unit or currency error — the classic XOF divide-by-100	Amount is `bigint` minor units with its currency code, copied wholesale from the payment. Nothing in the refund path performs arithmetic on it, so there is no place for an exponent mistake to enter.
Provider does not support refund status queries	*Cannot be mitigated in code.* If iPayMoney has no status endpoint, every `in_doubt` refund becomes a manual phone-and-statement process. §N item 3 exists so this is discovered before launch and not after the first ambiguous refund.

> **ONE FINANCIAL RISK I CANNOT DESIGN AWAY**
>
> Until the ledger records money movements (Phase 4 settlement), a refund's only financial record is the `refunds` table. That is *consistent* with Phase 2, which posts nothing to the ledger — but it means the double-entry books do not yet know refunds exist. Decision 3 is where you tell me whether that stands.

R

Files I expect to change

FILE	CHANGE
`db/migrations/0012_refund_execution.sql`	**New.** `refunds`, `refund_attempts`, `payment_events.refund_id`, settings
`db/schema/commerce.ts`	The same in Drizzle
`core/refunds/state.ts · record.ts · execute.ts · index.ts`	**New.** Machine, creation, execution and resolution
`core/refunds/refunds.test.ts`	**New.** The matrix in §O
`core/payments/provider.ts`	`getRefund()` optional; refund-shaped verified event
`core/payments/mock.ts`	Deterministic slow / ambiguous / silent refunds
`core/payments/ipaymoney.ts`	New methods that **throw**, as the rest do
`core/orders/payment.ts`	Create the `owed` record when an order becomes `refund_due` — same transaction
`core/ledger/commerce-reconcile.ts`	Nine refund detectors
`core/authz/permissions.ts · seed/reference.ts`	Three permissions, settings
`web: admin refunds page + 3 API routes`	Queue, execute, retry, manual reconciliation
`web/e2e/refunds.spec.ts`	**New.** Browser coverage
`docs/architecture/refund-policy.md`	**New.** The approved design as a living document

Not touched: `ledger/flows.ts`, `ledger/post.ts`, `money/`, `auth/`, `consent/`, `content/`, and the checkout and late-payment logic beyond the one insertion above.

5 Decisions I need from you

I have not taken any of these. Each has a recommendation and the consequence of choosing otherwise.

#	DECISION	OPTIONS	I RECOMMEND
1	**Who starts a refund?**	**A** automatic — the system executes as soon as one is owed · **B** record automatically, execute on an operator's action · **C** operator does both	**B.** The record is created automatically so nothing is ever lost; a human authorises the money leaving. At Afrinext's volume that is a few clicks a week, and it means no bug can drain the merchant account unattended. **A** is faster for the buyer and removes the human check.
2	**Partial refunds?**	**A** full-amount only · **B** support partial from the start	**A.** Every `refund_due` today is a full non-fulfilment. Partial refunds need a policy about what a partial entitlement is worth, which does not exist. The schema keeps an amount column, so **B** stays a later change, not a rewrite.
3	**Does a refund touch the ledger?**	**A** no — consistent with Phase 2, refunds live in `refunds` · **B** post a reversal now	**A.** Phase 2 posts nothing, so there is no capture to reverse; a refund entry with no matching capture would make the books *less* true. When settlement arrives it must record both. **B** would mean revisiting the approved ledger boundary now.
4	**Manual reconciliation — one person or two?**	**A** one finance user with step-up · **B** two-person rule, like withdrawals	**A** for now, with the audit trail. **B** matches the existing "nobody approves their own withdrawal" principle and is stronger, but Afrinext may not have two finance staff yet — a control nobody can satisfy gets worked around.
5	**How is the queue processed?**	**A** operator presses a button, no scheduler · **B** a domain function plus an authenticated endpoint an external scheduler calls · **C** build a worker daemon	**B.** The logic is testable and the scheduling is someone else's problem — a cron, a platform job, or an operator. **C** is real infrastructure this repository has none of, and it is not refund logic.
6	**Notification outbox now?**	**A** add an outbox table · **B** rely on the audit log later	**A.** One small table, no provider dependency, and buyer notification becomes a consumer rather than a redesign. **B** avoids a table but makes the audit log do a job it was not built for.
7	**Does `refund_due` stay a terminal order state?**	**A** yes — the order stays `refund_due` forever, refund state lives on the refund · **B** add `refunded` as a new terminal order state	**A.** One fact, one place. The order records that a refund is owed; the refund records what happened. **B** duplicates state across two tables and creates a new way for them to disagree.

Nothing is implemented. No file outside `docs/review/` has been modified, and I will not start until you approve — at which point I will build only the approved scope, stop, and produce the implementation review packet.

Phase 3 · refund execution · design gate · branch `claude/create-app-repository-1cxsih` · designed against code state `4342c06`